

MICROSERVICE 1 : SERVICE CONTAINERS

Objectif : Gérer les 2000 conteneurs intelligents avec capteurs IoT

ÉTAPES DE RÉALISATION

Phase 1 : Setup & Structure

1. Création du projet
 - Initialiser le dossier `service-containers`
 - Créer `package.json` avec les dépendances de base
 - Configurer l'environnement de développement
2. Configuration Supabase
 - Créer la table `containers` dans Supabase
 - Définir le schéma : `id`, `type`, `location`, `fill_level`, `capacity`, `iot_device_id`, `status`
 - Ajouter des données de test (10-20 conteneurs)
3. Connexion à la base de données
 - Configurer le client Supabase
 - Tester la connexion avec une requête simple
 - Gérer les erreurs de connexion

Phase 2 : API de Base

4. Endpoints CRUD
 - POST `/containers` → Créer un conteneur
 - GET `/containers` → Lister tous les conteneurs
 - GET `/containers/:id` → Récupérer un conteneur spécifique
 - PUT `/containers/:id` → Mettre à jour un conteneur
 - DELETE `/containers/:id` → Supprimer un conteneur
5. Fonctionnalités de recherche
 - GET `/containers?type=recyclage` → Filtrer par type
 - GET `/containers?zone=centre` → Filtrer par zone
 - GET `/containers?status=critical` → Filtrer par statut

- GET `/containers?fill_level>80` → Conteneurs presque pleins

Phase 3 : Données Géospatiales

6. Intégration PostGIS

- Activer l'extension PostGIS dans Supabase
- Modifier le champ `location` pour stocker des coordonnées GPS
- Créer des indexes spatiaux pour les performances

7. Endpoints géographiques

- GET `/containers/nearby?lat=...&lng=...&radius=...` → Conteneurs proches
- GET `/containers/in-zone?zone_id=...` → Conteneurs dans une zone
- Calcul des distances et tracés sur carte

Phase 4 : Temps Réel

8. WebSocket pour mises à jour

- Installer et configurer [Socket.io](https://socket.io/)
- Créer un endpoint WebSocket
- Émettre des événements quand un conteneur est mis à jour
- Gérer les connexions/déconnexions

Phase 5 : Fonctionnalités Avancées

9. Gestion des photos

- Configurer Supabase Storage
- POST `/containers/:id/photos` → Uploader une photo
- Stocker et récupérer les URLs des photos

10. Statistiques et monitoring

- GET `/containers/stats` → Statistiques globales
 - Calcul des moyennes, totaux, pourcentages
 - Historique des niveaux de remplissage
-

MICROSERVICE 2 : SERVICE USERS

Objectif : Gérer 15 000 citoyens, 50 agents, 10 gestionnaires

ÉTAPES DE RÉALISATION

Phase 1 : Authentification

1. Setup du projet
 - Créer le dossier `service-users`
 - Installer les dépendances nécessaires
 - Configurer les variables d'environnement
2. Intégration Supabase Auth
 - Utiliser l'authentification intégrée de Supabase
 - Configurer les providers (email, Google optionnel)
 - Tester l'inscription et la connexion

Phase 2 : Gestion des Profils

3. Table des utilisateurs
 - Créer la table `users` dans Supabase
 - Colonnes : id, email, username, role, points, badges, created_at
 - Lier avec la table `auth.users` de Supabase
4. Endpoints de base
 - POST `/auth/register` → Inscription utilisateur
 - POST `/auth/login` → Connexion (retourne JWT)
 - GET `/users/profile` → Récupérer le profil
 - PUT `/users/profile` → Mettre à jour le profil

Phase 3 : Rôles et Permissions

5. Système de rôles
 - Définir 4 rôles : citizen, agent, manager, admin
 - Créer un middleware d'autorisation
 - Vérifier le rôle sur chaque endpoint
6. Permissions par rôle
 - Citizen : signaler, consulter, participer
 - Agent : voir tournées, valider collectes

- Manager : créer tournées, voir statistiques
- Admin : tout

Phase 4 : Sécurité Avancée

7. Validation et sécurité
 - Valider tous les inputs utilisateur
 - Sanitizer les données
 - Implémenter rate limiting (100 req/min)
 - Journaliser les tentatives de connexion
8. Gestion des sessions
 - Refresh tokens
 - Logout et invalidation de token
 - Sessions simultanées limitées

Phase 5 : Fonctionnalités Sociales

9. Gestion des avatars
 - Upload d'avatar avec Supabase Storage
 - Redimensionnement automatique
 - Formats supportés : JPG, PNG, WebP
10. Notifications utilisateur
 - Table notifications dans Supabase
 - GET /users/notifications → Voir les notifications
 - PUT /users/notifications/:id/read → Marquer comme lu

MICROSERVICE 3 : SERVICE ROUTES

Objectif : Optimiser les tournées de collecte (-20% de distance)

ÉTAPES DE RÉALISATION

Phase 1 : Structure de Base

1. Setup du projet

- Créer le dossier `service-routes`
 - Installer les dépendances (Express, Supabase, algorithm libs)
 - Configurer la connexion à la BDD
2. Modélisation des données
- Table `routes` : id, date, agent_id, status, containers_list
 - Table `route_steps` : étape dans une tournée
 - Relations avec tables `containers` et `users`

Phase 2 : CRUD des Tournées

3. Endpoints de gestion
- POST `/routes` → Créer une nouvelle tournée
 - GET `/routes` → Lister toutes les tournées
 - GET `/routes/:id` → Détails d'une tournée
 - PUT `/routes/:id` → Modifier une tournée
 - DELETE `/routes/:id` → Supprimer une tournée
4. Assignment aux agents
- PUT `/routes/:id/assign` → Assigner à un agent
 - GET `/routes/agent/:agentId` → Tournées d'un agent
 - Changer le statut : `planned` → `in_progress` → `completed`

Phase 3 : Optimisation Algorithmique

5. Algorithme du voyageur de commerce (TSP)
- Récupérer les conteneurs à collecter
 - Calculer les distances entre conteneurs
 - Implémenter l'algorithme "nearest neighbor"
 - Générer l'ordre optimal de collecte
6. Amélioration avec 2-opt
- Prendre la solution initiale
 - Tester les échanges d'arêtes
 - Garder les améliorations
 - Répéter jusqu'à convergence

Phase 4 : Fonctionnalités Métier

7. Intégration conteneurs critiques
- Récupérer les conteneurs avec `fill_level > 80%`
 - Les inclure automatiquement dans les tournées

- Calculer l'économie de distance réalisée
- 8. Calculs temps et distance
 - Estimer le temps par collecte (5 min par conteneur)
 - Calculer la distance totale du trajet
 - Prévoir l'heure de fin de tournée

Phase 5 : Export et Visualisation

- 9. Feuille de route PDF
 - Générer un PDF avec l'itinéraire
 - Inclure carte, liste des conteneurs, heures estimées
 - Envoyer par email à l'agent
- 10. Visualisation sur carte
 - Endpoint GET `/routes/:id/map` → Données pour carte
 - Format GeoJSON pour affichage
 - Inclure popups avec infos conteneurs

MICROSERVICE 4 : SERVICE GAMIFICATION

Objectif : Engager les citoyens avec points, badges et défis

ÉTAPES DE RÉALISATION

Phase 1 : Système de Points

- 1. Setup du projet
 - Créer le dossier `service-gamification`
 - Configurer la connexion à Supabase
 - Définir la structure des données
- 2. Table des actions et points
 - Table `user_actions` : action, points, user_id, timestamp
 - Définir les valeurs : signalement=10pts, défi=50pts, etc.
 - Mettre à jour le total de points dans `users`

Phase 2 : Badges et Récompenses

3. Système de badges

- Table `badges` : id, name, description, icon, points_required
- Table `user_badges` : user_id, badge_id, awarded_at
- Badges : Débutner (100pts), Éco-Warrior (500pts), Super-Héros (1000pts)

4. Attribution automatique

- Vérifier quand un utilisateur atteint un seuil
- Attribuer le badge automatiquement
- Envoyer une notification

Phase 3 : Défis et Compétitions

5. Défis individuels et collectifs

- Table `challenges` : titre, description, objectif, récompense, période
- Table `challenge_participations` : progression, statut
- Types : "Signaler 10 conteneurs", "Zéro débordement quartier"

6. Classement (Leaderboard)

- GET `/gamification/leaderboard` → Top 50 utilisateurs
- Tri par points totaux
- Inclure rangs et badges

Phase 4 : Notifications Gamification

7. Alertes de progression

- Notifier quand un badge est gagné
- Rappels pour les défis en cours
- Félicitations pour les objectifs atteints

8. Statistiques personnelles

- GET `/gamification/users/:id/stats`
- Points par jour/semaine/mois
- Comparaison avec la moyenne
- Impact environnemental (CO2 économisé)

MICROSERVICE 5 : SERVICE ANALYTICS

Objectif : Tableaux de bord et rapports pour les gestionnaires

ÉTAPES DE RÉALISATION

Phase 1 : Données Agrégées

1. Setup et Data Warehouse
 - Créer le dossier `service-analytics`
 - Configurer les vues matérialisées dans Supabase
 - Structure en étoile : faits + dimensions
2. Collecte des données
 - Agrégations quotidiennes des mesures
 - Statistiques par zone, par type de conteneur
 - Performances des agents

Phase 2 : Tableaux de Bord

3. Dashboard temps réel
 - GET `/analytics/dashboard` → KPIs en direct
 - Nombre de conteneurs critiques
 - Tournées en cours
 - Signalements aujourd'hui
4. Visualisations
 - Graphiques avec Chart.js ou similar
 - Heatmaps des zones problématiques
 - Évolutions temporelles

Phase 3 : Rapports Automatisés

5. Génération de rapports
 - Rapports quotidiens pour les agents
 - Rapports hebdomadaires pour les gestionnaires
 - Rapports mensuels pour la direction
6. Formats d'export
 - PDF avec mise en page professionnelle
 - Excel pour analyse approfondie
 - Envoi automatique par email

Phase 4 : Prédictions ML

7. Prédiction de remplissage
 - Entraîner un modèle simple (régression)
 - Prédire quels conteneurs seront pleins demain
 - Intégrer les prévisions météo
8. Détection d'anomalies
 - Identifier les capteurs défectueux
 - Détecter les comportements anormaux
 - Alertes automatiques

MICROSERVICE 6 : SERVICE IOT

Objectif : Recevoir et traiter les données des 2000 capteurs

ÉTAPES DE RÉALISATION

Phase 1 : Réception des Données

1. Setup MQTT Broker
 - Installer Mosquitto ou utiliser service cloud
 - Configurer les topics : `containers/+/data`
 - Sécuriser avec TLS et authentification
2. Réception des messages
 - Service qui écoute le broker MQTT
 - Parse les messages JSON des capteurs
 - Valider le format des données

Phase 2 : Traitement et Stockage

3. Validation des données
 - Vérifier les plages (0-100% pour `fill_level`)
 - Détecter les valeurs aberrantes
 - Ignorer les données corrompues
4. Stockage en base
 - Insérer dans la table `measurements`
 - Mettre à jour le `fill_level` dans `containers`
 - Garder l'historique pour analytics

Phase 3 : Alertes Automatiques

5. Seuils d'alerte
 - Si fill_level > 90% → alerte critique
 - Si pas de données depuis 24h → alerte capteur
 - Si température anormale → alerte maintenance
6. Notifications
 - Envoyer aux gestionnaires concernés
 - Push notifications via service-users
 - Logs dans table `alerts`

API GATEWAY

Objectif : Point d'entrée unique pour tous les microservices

ÉTAPES DE RÉALISATION

Phase 1 : Structure de Base

1. Setup du projet
 - Créer le dossier `api-gateway`
 - Installer Express et http-proxy-middleware
 - Configurer les routes de proxy
2. Configuration des services
 - Définir les URLs de chaque microservice
 - Configurer le load balancing simple
 - Gérer les timeouts et retries

Phase 2 : Gestion des Requêtes

3. Routing intelligent
 - `/api/containers/*` → vers service-containers
 - `/api/users/*` → vers service-users
 - `/api/routes/*` → vers service-routes
 - etc.

4. Aggrégation de réponses
 - Pour certains endpoints, combiner des données de plusieurs services
 - Cache des réponses fréquentes
 - Formatage standardisé des réponses

Phase 3 : Sécurité et Monitoring

5. Sécurité centralisée
 - Validation JWT sur toutes les requêtes
 - Rate limiting global
 - Logs d'accès centralisés
6. Health checks
 - Vérifier que tous les services sont up
 - Endpoint `/health` avec statut de chaque service
 - Alertes si service down

DÉPLOIEMENT ET INTÉGRATION

ÉTAPES FINALES

1. Containerisation
 - Dockerfile pour chaque microservice
 - docker-compose.yml pour lancer tout l'environnement
 - Variables d'environnement par service
2. CI/CD Pipeline
 - Tests automatiques à chaque push
 - Build des images Docker
 - Déploiement sur serveur de test
3. Documentation
 - Documentation API (Swagger/OpenAPI)
 - Guide d'installation et déploiement
 - Manuel utilisateur
4. Tests de charge
 - Simuler 2000 capteurs envoyant des données
 - Tester avec 1000 utilisateurs simultanés
 - Optimiser les performances
5. Sécurité finale

- Scan de vulnérabilités
- Audit des permissions
- Backup et recovery plan